

100

Srihas Gupta

Other

PDF generated at: 15 Jun 2026 02:59:35 UTC

View this report on HackerRank

Score

100% • 80.0 / 80  
scored in CodePath TIP103: Unit 2 Assessment, Version A - Summer 2026 in 36 min 27 sec on 14 Jun 2026 19:21:07 PDT

Candidate Information

|                               |                                                             |
|-------------------------------|-------------------------------------------------------------|
| Email                         | srihasgupta@gmail.com                                       |
| Test                          | CodePath TIP103: Unit 2 Assessment, Version A - Summer 2026 |
| Candidate Packet              | <a href="#">View</a>                                        |
| Taken on                      | 14 Jun 2026 19:21:07 PDT                                    |
| Time taken                    | 36 min 27 sec/ 90 min                                       |
| Personal Member ID            | 154368                                                      |
| Email Address with CodePath   | srihasgupta@gmail.com                                       |
| Github username with CodePath | srihas115                                                   |
| Invited by                    | CodePath                                                    |

Skill Distribution



There is no associated skills data that can be shown for this assessment

Tags Distribution



There is no associated tags data that can be shown for this assessment

Questions

Coding Questions • 60 / 60

| No. | Question                     | Skill | Score | Code Quality | Time Taken   | Status |
|-----|------------------------------|-------|-------|--------------|--------------|--------|
| 1   | Array Intersection II Coding | -     | 20/20 | -            | 8 min        | ✔      |
| 2   | Word Pattern Coding          | -     | 20/20 | -            | 8 min 39 sec | ✔      |
| 3   | Grouped Anagrams Coding      | -     | 20/20 | -            | 6 min 28 sec | ✔      |

Multiple Choice + Debugging • 20 / 20

| No. | Question                                                                                                                    | Skill | Score | Code Quality | Time Taken   | Status |
|-----|-----------------------------------------------------------------------------------------------------------------------------|-------|-------|--------------|--------------|--------|
| 4   | Which of the following options will remove the key-value pair with key "grade" from the dictionary student? Multiple Choice | -     | 5/5   | -            | 2 min 1 sec  |        |
| 5   | Which of the following options will return the value 'B'? Multiple Choice                                                   | -     | 5/5   | -            | 5 min 37 sec |        |
| 6   | What is the output of the following code snippet? Multiple Choice                                                           | -     | 5/5   | -            | 2 min 32 sec |        |
| 7   | Find the bug! Coding                                                                                                        | -     | 5/5   | -            | 1 min 35 sec |        |

## 1. Array Intersection II

Correct

Coding

### Question description

Given two integer arrays `nums1` and `nums2`, return an array of their intersection. Your answer should be returned in **ascending order**.

Example 1:

Input: `nums1 = [1,2,2,1]`, `nums2 = [2,2]`

Expected Output: `[2,2]`

Example 2:

Input: nums1 = [4,9,5], nums2 = [9,4,9,8,4]  
Expected Output: [4,9]

## Candidate's Solution

Language used: Python 3

```
1  #!/bin/python3
2
3  import math
4  import os
5  import random
6  import re
7  import sys
8  import ast
9
10
11 #
12 # Complete the 'intersect' function below.
13 #
14 # The function is expected to return an INTEGER_ARRAY.
15 # The function accepts following parameters:
16 # 1. INTEGER_ARRAY nums1
17 # 2. INTEGER_ARRAY nums2
18 #
19
20 def intersect(nums1, nums2):
21     # Write your code here
22     freq_dict = {}
23
24     for num in nums1:
25         if num in freq_dict:
26             freq_dict[num] += 1
27         else:
28             freq_dict[num] = 1
29
30     res = []
31
32     for num in nums2:
33         if ((num in freq_dict) and (freq_dict[num] > 0)):
34             res.append(num)
35             freq_dict[num] -= 1
36
37     res.sort()
38
```

```

39     return res
40
41 if __name__ == '__main__':
42     outfile = open(os.environ['OUTPUT_PATH'], 'w')
43     input_data = sys.stdin.read().strip().splitlines()
44
45     for line in input_data:
46         if line == '':
47             continue
48
49         # Split the input and handle empty lists
50         nums1_str, nums2_str = line.split(',')
51         nums1_str = nums1_str.strip()[1:] # Remove leading '['
52         nums2_str = nums2_str.strip()[:-1] # Remove trailing ']'
53
54         # Handle empty lists
55         if nums1_str:
56             nums1 = list(map(int, nums1_str.split(',')))
57         else:
58             nums1 = []
59
60         if nums2_str:
61             nums2 = list(map(int, nums2_str.split(',')))
62         else:
63             nums2 = []
64
65         result = intersect(nums1, nums2)
66         outfile.write(str(result) + '\n')
67     outfile.close()

```

| TESTCASE   | DIFFICULTY | TYPE   | STATUS  | SCORE | TIME TAKEN | MEMORY USED |
|------------|------------|--------|---------|-------|------------|-------------|
| Testcase 0 | Easy       | Hidden | Success | 0     | 0.0329 sec | 10.9 KB     |
| Testcase 1 | Easy       | Hidden | Success | 0     | 0.0325 sec | 10.9 KB     |
| Testcase 2 | Easy       | Hidden | Success | 0     | 0.026 sec  | 10.8 KB     |

|             |      |        |         |   |               |         |
|-------------|------|--------|---------|---|---------------|---------|
| Testcase 3  | Easy | Hidden | Success | 0 | 0.0288<br>sec | 10.9 KB |
| Testcase 4  | Easy | Hidden | Success | 0 | 0.0294<br>sec | 10.9 KB |
| Testcase 5  | Easy | Hidden | Success | 0 | 0.0315<br>sec | 10.9 KB |
| Testcase 6  | Easy | Hidden | Success | 0 | 0.0329<br>sec | 10.9 KB |
| Testcase 7  | Easy | Hidden | Success | 0 | 0.0279<br>sec | 10.9 KB |
| Testcase 8  | Easy | Hidden | Success | 0 | 0.0258<br>sec | 10.8 KB |
| Testcase 9  | Easy | Hidden | Success | 0 | 0.0257<br>sec | 10.9 KB |
| Testcase 10 | Easy | Hidden | Success | 0 | 0.0336<br>sec | 10.9 KB |
| Testcase 11 | Easy | Hidden | Success | 0 | 0.0275<br>sec | 10.8 KB |
| Testcase 12 | Easy | Hidden | Success | 0 | 0.0275<br>sec | 10.9 KB |
| Testcase 13 | Easy | Hidden | Success | 0 | 0.0258<br>sec | 10.9 KB |

|                |      |        |         |    |            |         |
|----------------|------|--------|---------|----|------------|---------|
| Testcase 14    | Easy | Hidden | Success | 0  | 0.0261 sec | 10.9 KB |
| Pass/Fail Case | Easy | Hidden | Success | 20 | 0.0322 sec | 10.9 KB |

🚫 No comments.

## 2. Word Pattern

✓ Correct

Coding

### Question description

Given a `pattern` and a string `s`, return `True` if `s` follows the same pattern and `False` otherwise. Here **follow** means that each letter in the `pattern` corresponds to exactly one unique word in the string `s`, and each word in the string `s` corresponds to exactly one unique letter in the `pattern`. This means that:

1. Each letter in the `pattern` should map to the same word every time it appears.
2. Each word in the string `s` should map to the same letter every time it appears.

Example 1:

Input: `pattern = "abba"`, `s = "dog cat cat dog"`

Output: `True`

Example 2:

Input: `pattern = "abba"`, `s = "dog cat cat fish"`

Output: `False`

Explanation:

The initial mapping is:

'a' → "dog"

'b' → "cat"

However, when the pattern repeats:

The second 'a' should map to "dog" (as 'a' is already mapped to "dog"), but instead, it would map to "fish".

This is a violation because:

'a' cannot map to both "dog" and "fish".  
Hence, the mapping is inconsistent.  
Therefore, the output is False.

## Candidate's Solution

Language used: Python 3

```
1  #!/bin/python3
2
3  import math
4  import os
5  import random
6  import re
7  import sys
8  import ast
9
10
11 #
12 # Complete the 'word_pattern' function below.
13 #
14 # The function is expected to return a BOOLEAN.
15 # The function accepts following parameters:
16 # 1. STRING pattern
17 # 2. STRING s
18 #
19
20 def word_pattern(pattern, s):
21     # Write your code here
22     words_arr = s.split()
23
24     if (len(pattern) != len(words_arr)):
25         return False
26
27     char_to_word = {}
28     word_to_char = {}
29
30     for i in range(len(pattern)):
31         char = pattern[i]
32         word = words_arr[i]
33
```



```

34         if char in char_to_word:
35             if char_to_word[char] != word:
36                 return False
37         else:
38             if word in word_to_char:
39                 return False
40             char_to_word[char] = word
41             word_to_char[word] = char
42
43     return True
44
45
46 if __name__ == '__main__':
47     outfile = open(os.environ['OUTPUT_PATH'], 'w')
48     input_data = sys.stdin.read().strip().splitlines()
49
50     for line in input_data:
51         # Skip empty or improperly formatted lines
52         if not line.strip():
53             continue
54
55         # Safely split the line
56         try:
57             pattern_str, sentence = line.split(",")
58             pattern_str = pattern_str.strip().strip('\'')
59             sentence = sentence.strip().strip('\'')
60         except ValueError:
61             print("Invalid input format:", line)
62             continue
63
64         result = word_pattern(pattern_str, sentence)
65         outfile.write(str(result) + '\n')
66     outfile.close()
67

```

| TESTCASE   | DIFFICULTY | TYPE   | STATUS  | SCORE | TIME TAKEN | MEMORY USED |
|------------|------------|--------|---------|-------|------------|-------------|
| Testcase 0 | Easy       | Hidden | Success | 0     | 0.0275 sec | 10.9 KB     |

|                                                            |      |        |         |   |            |         |
|------------------------------------------------------------|------|--------|---------|---|------------|---------|
| Testcase 1                                                 | Easy | Hidden | Success | 0 | 0.0261 sec | 10.9 KB |
| Testcase 2                                                 | Easy | Hidden | Success | 0 | 0.0268 sec | 10.9 KB |
| Empty Pattern and Empty String                             | Easy | Hidden | Success | 0 | 0.0264 sec | 10.9 KB |
| Pattern with Single Character and Single Word              | Easy | Hidden | Success | 0 | 0.0254 sec | 10.9 KB |
| Pattern with Repeated Characters and Single Word           | Easy | Hidden | Success | 0 | 0.0296 sec | 10.9 KB |
| Pattern Longer Than String                                 | Easy | Hidden | Success | 0 | 0.03 sec   | 10.9 KB |
| String Longer Than Pattern                                 | Easy | Hidden | Success | 0 | 0.0251 sec | 10.9 KB |
| Pattern and String of Equal Length with Different Words    | Easy | Hidden | Success | 0 | 0.0271 sec | 10.9 KB |
| Pattern and String with All Different Characters and Words | Easy | Hidden | Success | 0 | 0.0271 sec | 10.9 KB |

|                                                   |      |        |         |    |            |         |
|---------------------------------------------------|------|--------|---------|----|------------|---------|
| Pattern with Mixed Characters and Different Words | Easy | Hidden | Success | 0  | 0.0262 sec | 10.9 KB |
| Pass/Fail Case                                    | Easy | Hidden | Success | 20 | 0.0274 sec | 10.9 KB |

🚫 No comments.

### 3. Grouped Anagrams

✅ Correct

Coding

#### Question description

Given a list of strings `strs`, group **the anagrams** together. You can return the answer in **any order**. An **anagram** is a word or phrase formed by rearranging the letters of a different word or phrase, using each of the original letters exactly once.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Expected Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Answers in a different order such as `[["tan", "nat"], ["bat"], ["ate", "tea", "eat"]]` would also be acceptable

Example 2:

Input: `strs = [""]`

Expected Output: `[[""]]`

Example 3:

Input: `strs = ["a"]`

Expected Output: `[["a"]]`

## Candidate's Solution

Language used: Python 3

```
1  #!/bin/python3
2
3  import math
4  import os
5  import random
6  import re
7  import sys
8  import ast
9
10
11
12  #
13  # Complete the 'grouped' function below.
14  #
15  # The function is expected to return a STRING_ARRAY.
16  # The function accepts STRING_ARRAY strs as parameter.
17  #
18
19  def grouped_anagrams(strs):
20      # Write your code here
21      anagram_map = {}
22
23      for word in strs:
24          sorted_chars_arr = sorted(word)
25          key = "".join(sorted_chars_arr)
26
27          if key not in anagram_map:
28              anagram_map[key] = []
29
30          anagram_map[key].append(word)
31
32      return list(anagram_map.values())
33
34  if __name__ == '__main__':
35      outfile = open(os.environ['OUTPUT_PATH'], 'w')
36      input_lines = sys.stdin.read().strip().splitlines()
37
38      for input_str in input_lines:
39          if input_str.strip() == "":
40              continue
41
42          try:
43              strs = ast.literal_eval(input_str)
44
```

```

45     result = grouped_anagrams(strs)
46
47     result = [sorted(group) for group in result]
48     result = sorted(result, key=lambda x: x[0] if x else '')
49
50     outfile.write(str(result) + '\n')
51 except (ValueError, SyntaxError):
52     print("Error: Invalid input")
53 outfile.close()

```

| TESTCASE   | DIFFICULTY | TYPE   | STATUS  | SCORE | TIME TAKEN | MEMORY USED |
|------------|------------|--------|---------|-------|------------|-------------|
| Testcase 0 | Easy       | Hidden | Success | 0     | 0.0289 sec | 10.9 KB     |
| Testcase 1 | Easy       | Hidden | Success | 0     | 0.0291 sec | 10.9 KB     |
| Testcase 2 | Easy       | Hidden | Success | 0     | 0.0267 sec | 11 KB       |
| Testcase 3 | Easy       | Hidden | Success | 0     | 0.0255 sec | 10.9 KB     |
| Testcase 4 | Easy       | Hidden | Success | 0     | 0.0303 sec | 10.9 KB     |
| Testcase 5 | Easy       | Hidden | Success | 0     | 0.026 sec  | 10.9 KB     |
| Testcase 6 | Easy       | Hidden | Success | 0     | 0.0276 sec | 10.9 KB     |
| Testcase 7 | Easy       | Hidden | Success | 0     | 0.0265 sec | 10.9 KB     |

Pass/Fail  
Case

Easy

Hidden

Success

20

0.0262  
sec

11 KB

! No comments.

**4. Which of the following options will remove the key-value pair with key "grade" from the dictionary student?**

✓ Correct

Multiple Choice

### Question description

Which of the following options will remove the key-value pair with key "grade" from the dictionary student?

```
student = {  
    "name": "Emma",  
    "class": 9,  
    "grade": 'A'  
}
```

### Candidate's Solution

Options: (Expected answer indicated with a tick)



`<p><span style="font-family:Courier  
New,Courier,monospace;">student.pop("&quot;grade&quot;")</span></p>`



<p><span style="font-family:Courier

☐ New,Courier,monospace;">student.clear(&quot;grade&quot;)</span><!-- notionvc:  
78481de2-34d1-43a5-8346-cdb85fd739d2 --></p>

<p><span style="font-family:Courier

☐ New,Courier,monospace;">student.remove(&quot;grade&quot;)</span><!-- notionvc:  
9184348c-01f1-4772-b551-fd097b6cfcb9 --></p>

<p><span style="font-family:Courier

☐ New,Courier,monospace;">student.popitem(&quot;grade&quot;)</span><!-- notionvc:  
674db0e2-5aea-49e8-bc07-ddfbd38bd28a --></p>

⚠ No comments.

## 5. Which of the following options will return the value 'B'?

✓ Correct

Multiple Choice

### Question description

```
gradebook = {  
  "class":{  
    "student":{  
      "name":"Mike",  
      "grade":{  
        "physics":'C',  
        "history": 'B'  
      }  
    }  
  }  
}
```

```
}  
}
```

## Candidate's Solution

Options: (Expected answer indicated with a tick)



```
<p><span style="font-family:Courier  
New,Courier,monospace;">gradebook[&#39;class&#39;][&#39;student&#39;]  
[&#39;grade&#39;][&#39;history&#39;]</span><!-- notionvc: 021c1267-aa34-4faf-94d0-  
30ed10966180 --></p>
```



```
<p><span style="font-family:Courier  
New,Courier,monospace;">gradebook[&#39;class&#39;][&#39;student&#39;]  
[&#39;grade&#39;][1]</span><!-- notionvc: 2f70ec77-30df-4a85-9ce7-9963fe1b0db6 -->  
</p>
```



```
<p><span style="font-family:Courier  
New,Courier,monospace;">gradebook[&#39;class&#39;][0][&#39;grade&#39;]  
[&#39;history&#39;]</span><!-- notionvc: 37b920a1-1d7b-47d6-b68d-76ff1a325f1a -->  
</p>
```



```
<p><span style="font-family:Courier  
New,Courier,monospace;">gradebook[&#39;class&#39;][&#39;student&#39;][1]  
[&#39;history&#39;]</span><!-- notionvc: 01906bd6-ba2a-4802-8ac7-494a4a260a49 -->  
</p>
```



⚠ No comments.

## 6. What is the output of the following code snippet?

✓ Correct

Multiple Choice

### Question description

```
def process_data(names, scores):
    result = {}
    for i in range(len(names)):
        name = names[i]
        score = scores[i]
        if name not in result:
            result[name] = []
        result[name].append(score)
    return result

names = ["Alice", "Bob", "Alice", "Bob", "Charlie"]
scores = [85, 90, 95, 80, 70]
result = process_data(names, scores)
print(result)
```

### Candidate's Solution

Options: (Expected answer indicated with a tick)

☒ `<p><span style="font-family:Courier New,Courier,monospace;">{'Alice': [85, 95], 'Bob': [90, 80], 'Charlie': [70]}`</span><!-- notionvc: 2aa562bd-2e5b-4c67-be76-86b0b705bd5f --></p>





```
<p><span style="font-family:Courier New,Courier,monospace;">{&#39;Alice&#39;; [85, 90, 95, 80, 70]}</span></p>
```



```
<p><span style="font-family:Courier New,Courier,monospace;">{&#39;Alice&#39;; [85, 95], &#39;Bob&#39;; [90], &#39;Charlie&#39;; [70]}</span><!-- notionvc: 78225981-972f-4022-97a8-505d65191dab --></p>
```



```
<p><span style="font-family:Courier New,Courier,monospace;">{&#39;Alice&#39;; [95], &#39;Bob&#39;; [80], &#39;Charlie&#39;; [70]}</span><!-- notionvc: c0956312-cbc6-40cd-9277-7e820f4ad602 --></p>
```

⚠ No comments.

## 7. Find the bug!

✓ Correct

Coding

### Question description

You are given a function `filter_below_threshold` that is intended to take two inputs:

- A dictionary `dict1` where the keys are associated with integer values.
- An integer `threshold`

The goal of the function is to return a new dictionary that contains only the key-value pairs from `dict1` where the values are **strictly more than** the given `threshold`.

However, the current implementation of the function contains one or more bugs and does not pass the test cases as expected. Your task is to identify and correct the issues in the implementation so that the function works as intended.

## Candidate's Solution

Language used: Python 3

```
1  #!/bin/python3
2
3  import math
4  import os
5  import random
6  import re
7  import sys
8  import ast
9
10
11 #
12 # Complete the 'filter_below_threshold' function below.
13 #
14 # The function is expected to return an INTEGER_ARRAY.
15 # The function accepts following parameters:
16 # 1. INTEGER_ARRAY dict1
17 # 2. INTEGER threshold
18 #
19
20 def filter_below_threshold(dict1, threshold):
21     filtered_dict = {}
22     for key, value in dict1.items():
23         if value > threshold:
24             filtered_dict[key] = value
25     return filtered_dict
26
27 if __name__ == '__main__':
28     import sys
29     import ast
30     import io
31
32     # Capture original stdout
33     original_stdout = sys.stdout
34     sys.stdout = io.StringIO() # Redirect stdout to a temporary buffer
35
36     try:
37         input_data = sys.stdin.read().strip()
38         input_list = ast.literal_eval(input_data)
39
40         dict1 = input_list[0]
41         threshold = input_list[1]
42
43         result = filter_below_threshold(dict1, threshold)
44
45     finally:
```

```
45 sys.stdout = original_stdout # Restore original stdout
46
47 print(result) # This is the ONLY output the system reads
48
```

| TESTCASE       | DIFFICULTY | TYPE   | STATUS  | SCORE | TIME TAKEN | MEMORY USED |
|----------------|------------|--------|---------|-------|------------|-------------|
| Pass/Fail Case | Easy       | Hidden | Success | 5     | 0.0258 sec | 11 KB       |

 No comments.